DIGITAL EGYPT PIONEERS INITIATIVE / DEVOPS TRACK

AdaptiveOps — Automated E-Commerce Deployment Platform

System Analysis & Design

Project 4 • Official submission date: 6 November 2026

The design defines how the team will build, deploy, observe and recover Google Online Boutique on local Ubuntu virtual machines. The approved October requirements govern the design. Runtime acceptance requires measured evidence from the implementation phase.

Project team

Team member	Primary responsibility
Amer Hossam Eldin Mohamed Mustafa (Team leader)	Jenkins CI/CD, release integration and automatic rollback
Kirollos Magdy Zekry Awad	Kubernetes, Nginx, HPA and networking
Rana Ashraf Abu Elyazeed Abu Elyazeed	Ansible, Terraform and feasible AWS infrastructure
Rahma Mohamed Abdo El-Sharqawy	Prometheus, Grafana, service metrics and KPIs
Jana Mohamed Mahmoud El-Waleely	Online Boutique feasibility, Docker builds and startup
Abdelrahman Ahmed Abdelbaset Mohamed	Smoke, load and rollback testing; execution documentation

Design edition 1.0 | Source review: 5 November 2026

READER'S GUIDE

Contents and evidence status

The release decision is the organizing principle of this design: a change is accepted only after the deployed revision passes functional checks. Application behavior, platform design and future test results have different evidence requirements.

Section	Subject	Pages
1	Project scope and approved baseline	3
2	Actors and use cases	4
3	Platform and application architecture	5–6
4	Data model, physical storage and code structure	7–8
5	Data flows, checkout and API contracts	9–11
6	Release gates, recovery and state transitions	12–14
7	Storefront and operator interface designs	15–16
8	Deployment, scaling, configuration and cloud	17–19
9	Testing, validation and acceptance evidence	20–21
10	Team ownership, milestones and unresolved decisions	22–23
A	DEPI Section 4 compliance checklist	24
B	Requirements, stories, use cases and deliverable crosswalk	25–29
C	References and source inventory	30–31

Evidence terms

Verified means the cited file or repository object was inspected. Approved plan identifies a binding October commitment. Proposed design identifies a mechanism or placement recommended for implementation. Not yet tested means that runtime evidence was not supplied. Requires clarification is reserved for an unresolved interpretation, such as the formal ERD format.

1 / SYSTEM ANALYSIS

Problem, objectives and approved scope

E-commerce releases can disrupt several dependent services at once. A running web process can conceal a broken catalog or checkout. AdaptiveOps addresses this release risk with a reproducible deployment, visibility into workload behavior and an explicit route back to a working revision.

The assignment is Project 4 — Automated E-Commerce Deployment Platform [7]. AdaptiveOps is its submitted name. The existing Google Online Boutique supplies the shop; the team owns the deployment and operations work. The technical scope of the former project of the same name is excluded.

Table 01 Audit of the approved October baseline

Approved document	Finding against DEPI and the Project 4 brief
Proposal [1], pp. 1–3	Defines the objective, existing app, six owners, tools and seven implementation weeks. Local first; Terraform for a feasible AWS subset.
Planning [2], pp. 1–2	Preserves the schedule, responsibilities, risks and six proposed KPIs. Deployment repetition, load, rollback and configuration each need observed evidence.
Literature and lecturer review [3], pp. 1–2	Records approval for Project 4, application reuse and feasible AWS. Search and storage limits are explicit; no numerical marking rubric was supplied.
Requirements [4], pp. 1–3	Defines 17 functional and 10 non-functional requirements, nine stories and three use cases. Appendix B preserves their IDs and acceptance intent.

Scope and acceptance

The nine Project 4 deliverables are Dockerized services; Jenkins update pipelines; Kubernetes with autoscaling; Ansible setup; Prometheus-backed monitoring dashboards; Nginx routing; Terraform scripts for feasible AWS resources; automatic rollback; and the assigned Git repository. Appendix B links each deliverable to its requirements.

The design targets two clean deployments, a controlled build/test/release, measured scale-out and reduction, live dashboard evidence, one intentionally faulty release with automatic recovery, and a repeatable Ansible run. Performance thresholds follow feasibility measurements [2, 4].

All four October files matched the Git blob identifiers of the official repository on 5 November [16]. The match establishes document identity; it does not establish implementation or test progress.

2 / FUNCTIONAL ANALYSIS

Actors and approved use cases

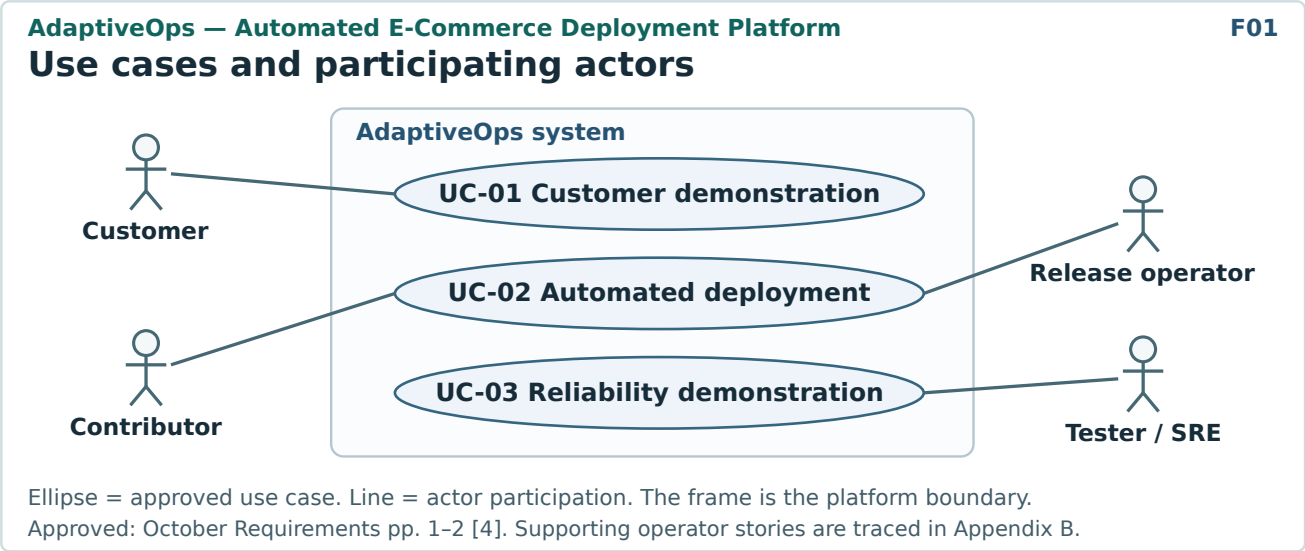


Figure 01 • The customer, contributor, release operator and tester participate in the three approved scenarios. Monitoring and infrastructure operators support these scenarios through dashboard and setup stories; they do not create new storefront functions.

Table 02 Use-case descriptions and outcomes

Use case and precondition	Main flow, outcome and exception
UC-01 Customer demonstration **Reachable storefront and test session**	Browse a product, add it to the same cart and simulate checkout. Confirm the order response and mock payment. A missing item or failed checkout fails the demonstration even if the health URL responds.
UC-02 Automated deployment **Reviewed revision and available target**	Jenkins tests, builds, publishes and deploys the identified revision. Rollout and smoke checks must pass before acceptance. Pre-deployment failure stops the build; a failed deployed update follows F10.
UC-03 Reliability demonstration **Valid metric and accepted prior release**	Apply controlled load, observe eligible replicas rise and return toward minimum, then inject a faulty release. Capture detection, automatic restore and functional recovery; failed recovery is recorded as blocked.

US-02 belongs to the tester: catalog search is exercised through gRPC. US-09 belongs to the infrastructure operator: Ansible and conditional Terraform support repeatable setup [4].

3.1 / SOFTWARE ARCHITECTURE

Boundaries of the deployment platform

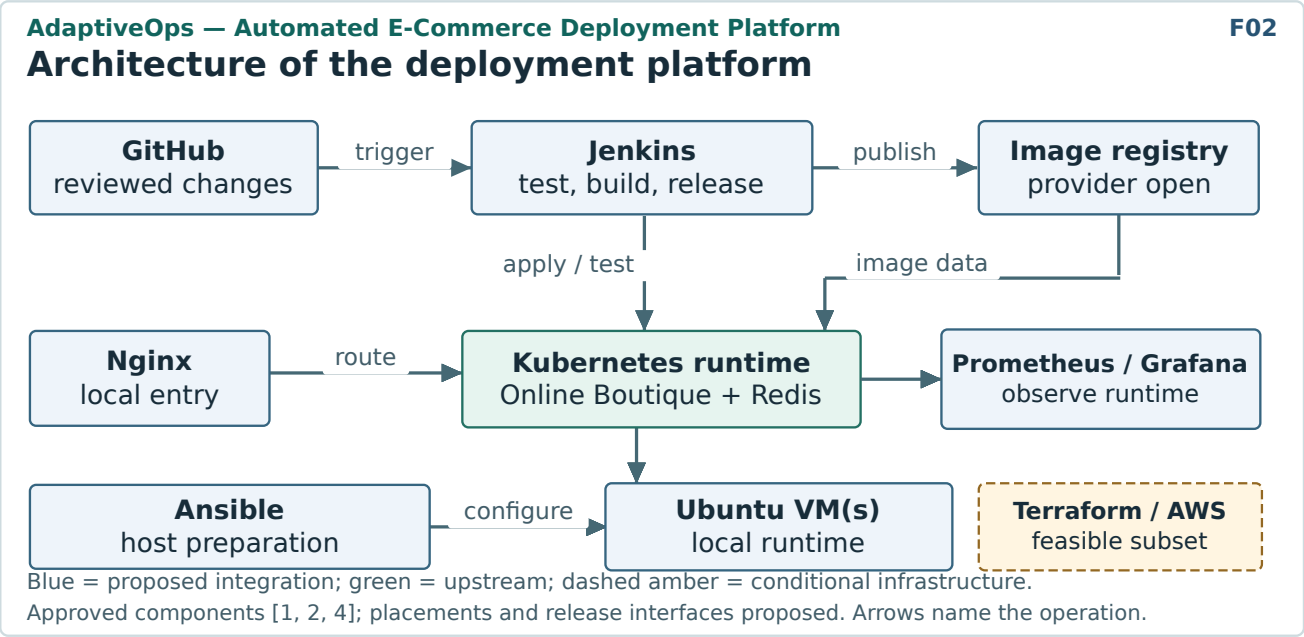


Figure 02 • The reviewed Git revision, image digest and deployment configuration travel together through the release plane. Runtime traffic enters through Nginx. The cloud branch has no dependency in the local shop or its recovery path.

Table 03 Technology stack and responsibility boundaries

Plane	Technology and responsibility
Application	The existing polyglot microservices communicate mainly through gRPC; the Go frontend serves HTTP. Redis holds cart state; the default catalog reads a JSON file [8–13].
Development and release	GitHub review; Jenkins test, Docker build and image publication; a registry reachable by both build agent and cluster. Jenkins deploys and records the accepted descriptor.
Runtime and routing	Ubuntu guest, local Kubernetes, Deployments/Services, configured Nginx and a valid HPA metric path. The application API stays internal.
Observation and recovery	Prometheus scrapes actual endpoints; Grafana queries it. Jenkins uses rollout and functional evidence to accept or restore a release. Grafana is not a deployment controller.
Configuration and cloud	Ansible configures the host. Terraform describes feasible AWS resources; live provisioning is conditional on permission and cost.

The architecture combines an existing microservice application with a versioned release process. Replacing a service image is a deployable change; changing persistent data is a different operation and is outside release rollback.

3.2 / SOFTWARE ARCHITECTURE

Online Boutique components and dependencies

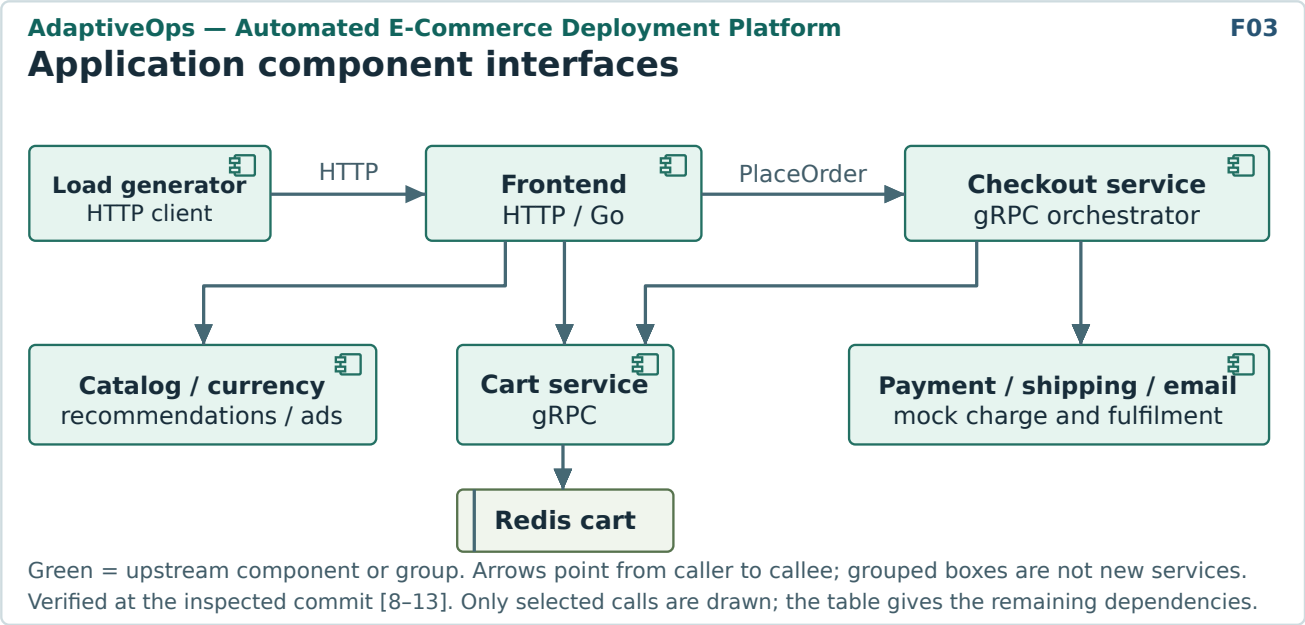


Figure 03 ● The upstream inventory counts ten shop services plus the load generator as eleven components; Redis is a separate dependency. Grouped boxes summarize callers and callees. The following table specifies the dependencies omitted from the drawing for readability.

Table 04 Upstream component inventory

Component and implementation	Verified interface or dependency at the inspected revision [8–13]
Frontend ● Go	Serves HTTP and calls catalog, cart, checkout, currency, shipping, recommendations and ads. Automatically assigned visitor sessions identify carts.
Catalog ● Go; cart ● C#	Catalog loads product documents and serves List/Get/Search. Cart delegates Add/Get/Empty to its store. The default manifest selects Redis via REDIS_ADDR.
Checkout ● Go	Reads cart; retrieves products and converts prices; gets a shipping quote; invokes mock payment, shipment, cart emptying and email confirmation.
Payment ● Node.js; shipping ● Go; email ● Python	Payment validates synthetic card data and returns a transaction ID; shipping supplies quotes and a mock tracking ID; email handles the simulated confirmation.
Currency ● Node.js; recommendations ● Python; ads ● Java	Currency converts amounts. Recommendation queries the catalog for product IDs. Ads supplies contextual messages to the frontend.
Load generator ● Python/Locust; Redis	Locust sends shopper-like HTTP requests. Redis serves cartservice on the internal network; it is neither an order database nor a customer endpoint.

The catalog, cart, checkout and mock payment path is in the assessed scope. Upstream optional variants and unrelated application extensions are not design dependencies.

4.1 / DATABASE DESIGN AND DATA MODELING

Logical entities and physical storage

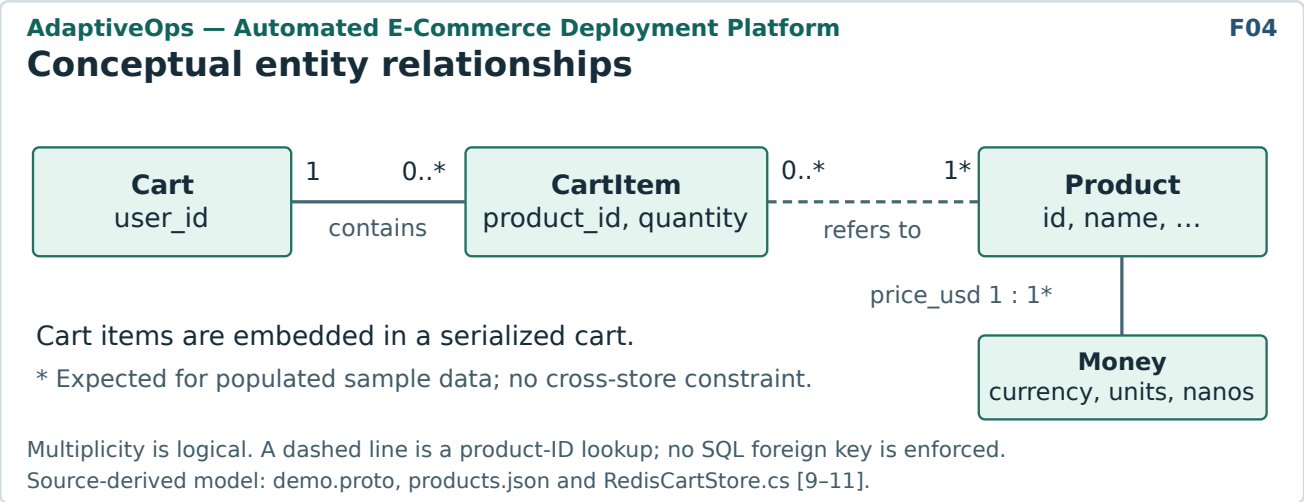


Figure 04 • A cart embeds zero or more items. Each item carries the intended catalog product ID and a quantity; it does not own a product row. Products in the reference catalog have a money value. This is a conceptual ER diagram of the actual data, not a relational database design.

Table 05 Logical to physical schema mapping [9–14]

Logical structure and identifier	Physical representation and limit
Product: string id, name, description and picture; Money price_usd; repeated string categories	The default loader reads products.json into protobuf products. JSON uses priceUsd with currencyCode, units and nanos. GetProduct scans for id; SearchProducts scans name and description. No SQL index.
Cart: string user_id, repeated CartItem; item: string product_id, int32 quantity	The cart service uses the session/user ID as its cache lookup key and serializes a protobuf Cart into bytes. A repeated product increments quantity. Redis is selected by the default manifest.
Money and order response	Money embeds a currency string, int64 units and int32 nanos. Checkout constructs an OrderResult containing order_id, shipping_tracking_id, shipping cost/address and items, then returns it. No orders table is present in this path.
Redis service and data volume	redis-cart:6379; /data is mounted on emptyDir. The volume can outlive a container restart in the same Pod, but is removed when that Pod is removed. It does not provide durable orders or carts.

There are no SQL tables, primary/foreign-key constraints or normal forms to declare. The product ID is an application reference across two stores. DEPI acceptance of this non-relational presentation in lieu of a conventional physical ERD is recorded as a format exception (§10.2).

4.2 / DATABASE DESIGN AND DATA MODELING

Code structure and the cart lifecycle

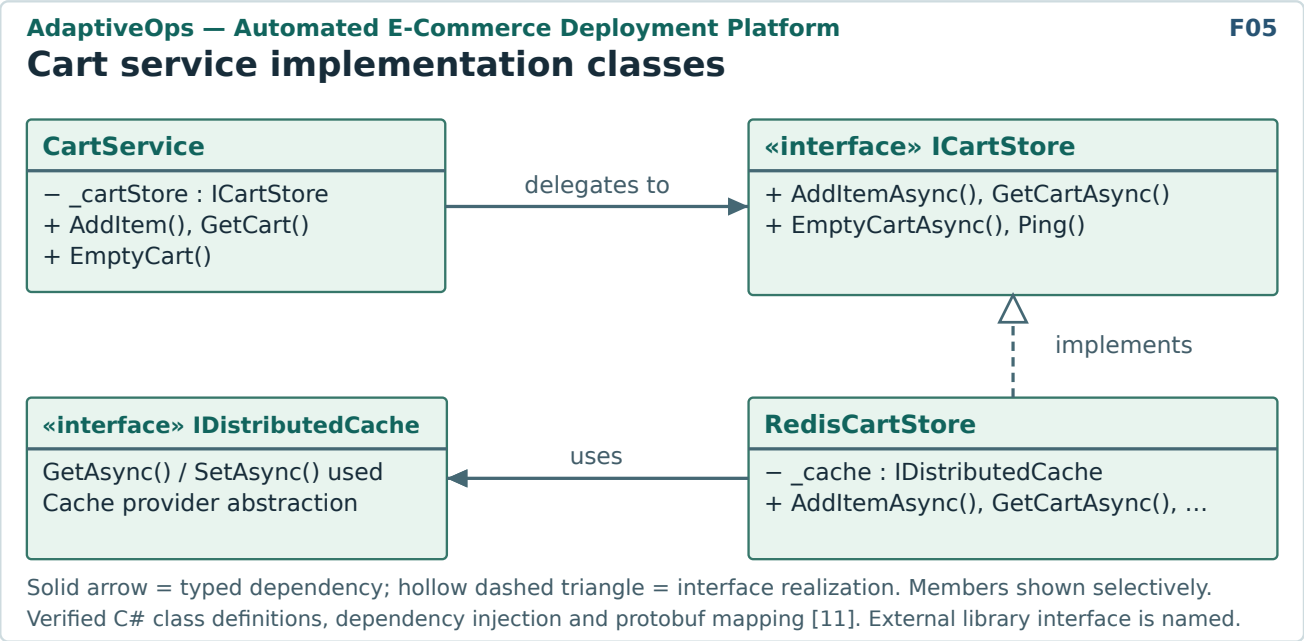


Figure 05 ● The C# gRPC adapter receives an ICartStore in its constructor. RedisCartStore realizes that interface and delegates byte I/O to IDistributedCache. With REDIS_ADDR set, Startup registers the Redis-backed implementation. The diagram models inspected classes rather than treating DevOps tools as application classes [11].

How a cart changes

AddItemAsync reads the cached bytes using the user ID. On a miss it creates a cart and inserts the product and quantity. On a hit it parses the cart, appends an item or increments an existing quantity, then writes the serialized cart back. A missing cart on GetCartAsync yields an empty cart. EmptyCartAsync writes an empty cart value rather than deleting the cache key.

Implications for verification

The approved cart smoke test must reuse the same session. A separate test session isolates another run. A Redis TCP probe and the static frontend health response do not establish that an item can be written and read. Recovery evidence must include an actual cart operation and a new mock checkout.

After Redis Pod replacement, the default volume cannot be assumed to retain cart contents. Restoring an application image/configuration cannot restore lost cart data or reverse payment, shipping or email calls. Only synthetic transactions are used.

5.1 / DATA FLOW AND SYSTEM BEHAVIOR

System context and boundary exchanges

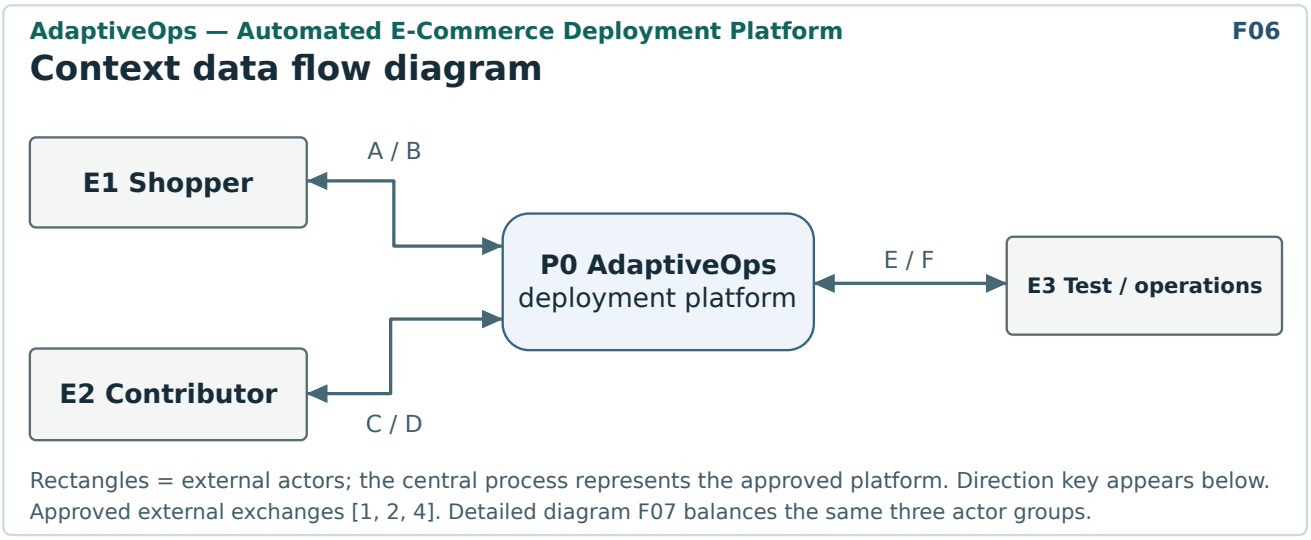


Figure 06 • The context diagram treats the shop and its delivery and observation tools as one system. The paired letters identify separate inbound and outbound data flows. This view covers the local demonstration; optional AWS provisioning is handled separately in §8.3.

Table 06 Data dictionary for the context diagram

External entity	Inbound to P0	Outbound from P0
E1 Shopper	A • Product selections, item/quantity, session cookie and synthetic checkout details.	B • Catalog pages, cart contents and the simulated order confirmation.
E2 Contributor / release operator	C • Reviewed Git change, source revision and controlled release trigger.	D • CI outcome, release digest/config reference, gate and recovery records.
E3 Tester / operations	E • Catalog query, load profile, observation query and controlled fault-test specification.	F • Matched products, client outcomes, metric samples, replica progression and test evidence.

A request to change a release and a shopper's request to buy an item carry different information and enter different interfaces. The contributor supplies source/configuration through GitHub. The shopper supplies only existing storefront inputs; administrative credentials are not carried on that path.

In F07 the same actor groups and six boundary flows reappear. Internal stores and measurements are expanded there; no new external actor, product capability or database is introduced.

5.2 / DATA FLOW AND SYSTEM BEHAVIOR

Detailed data flows and stores

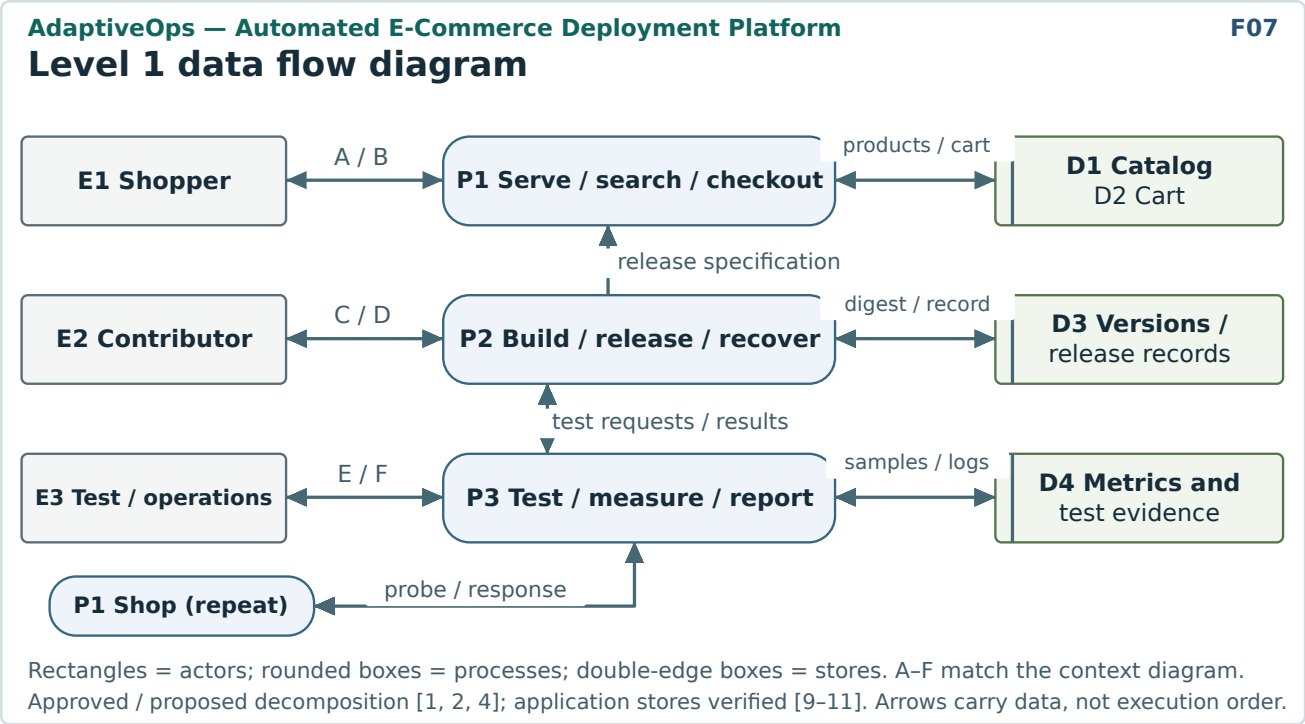


Figure 07 ● P1 reads catalog data and reads/writes cart contents. P2 exchanges image/configuration references with the release store. P3 exercises P1, returns gate outcomes to P2, and retains observations at D4. The repeated P1 symbol denotes the same process.

Table 07 Detailed process and store definitions

Process or store	What crosses the interface
P1 ● Serve and search	Browser requests use HTTP through Nginx. An internal test client calls SearchProducts by gRPC; it does not pass a query through a public search page.
P2 ● Build, release, recover	A reviewed commit yields an exact image digest and rendered config revision. The release specification selects the active application; gate decisions read results from P3.
P3 ● Test and observe	Test inputs exercise the deployment; client responses and timestamps become evidence. Prometheus time series and load-driver output are different records.
D1–D4 ● Storage boundaries	D1 catalog JSON is read-only in this flow; D2 Redis cart is read/write. D3 represents versioned artifacts/records. D4 groups Prometheus samples and sanitized evidence files logically, not as a new database.

Load measurements and incident logs do not become shop data. Evidence identifies the test run, release and time window; demo card values, secrets and complete session identifiers are excluded.

5.3 / INTERFACES AND SYSTEM BEHAVIOR

Checkout sequence and API contracts

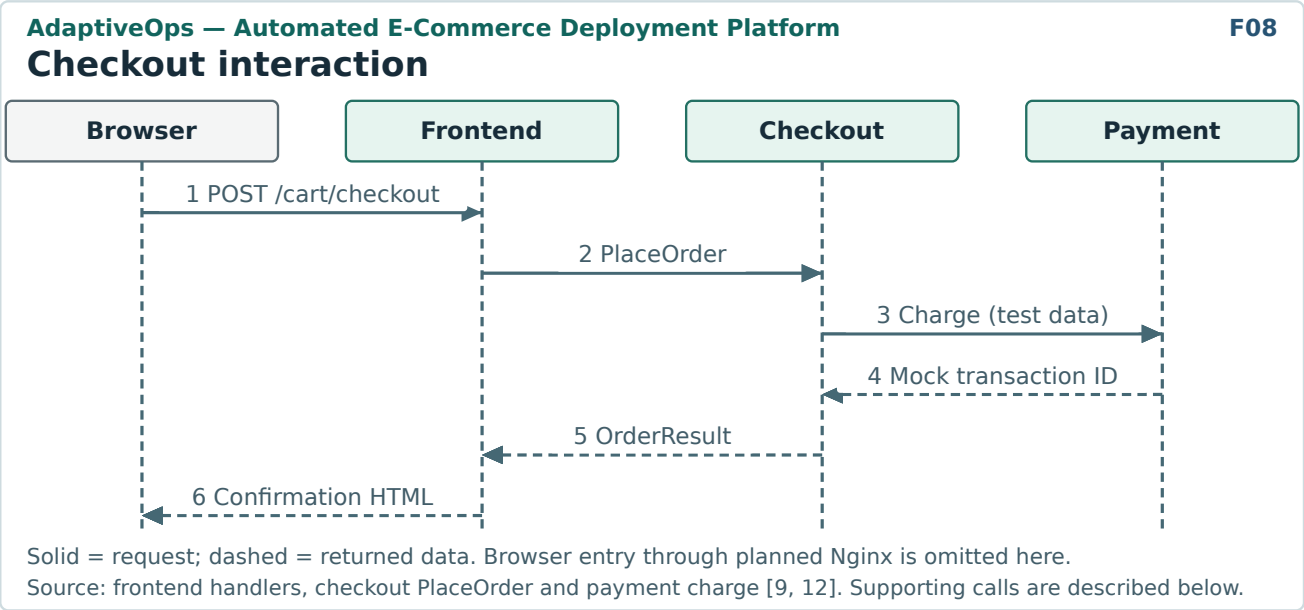


Figure 08 ● The browser submits the existing checkout form. Checkout reads the cart, resolves item prices and shipping, then calls PaymentService.Charge. After a mock charge it requests shipment, attempts to empty the cart and sends the confirmation. The response contains the order result; email failure is logged, and failure to empty the cart does not block return in this source [12].

Table 08 Interfaces used for functional acceptance

Interface	Input → outcome; verification use [9–13]
HTTP ● frontend via Nginx	GET / and /product/{id} → rendered products. POST /cart with product_id and quantity → cart update; GET /cart must show that item in the same session.
HTTP ● checkout / health	POST /cart/checkout with synthetic address, email, currency/session context and test card fields → order page. GET /_healthz → static “ok”; unsuitable as the sole release gate.
gRPC ● productcatalogservice:3550	hipstershop.ProductCatalogService/SearchProducts: {query: "sunglasses"} → results containing product OLJCESPC7Z in the inspected fixture. A controlled non-match yields zero products.
gRPC ● cart / checkout / payment	CartService.AddItem(user_id, item) → Empty; GetCart(user_id) → Cart. CheckoutService.PlaceOrder → PlaceOrderResponse.order. PaymentService.Charge(amount, credit_card) → transaction_id; mock only.

Requests use the deployed base URL, if configured. A test client uses the pinned proto inside the cluster or through a controlled port-forward; RPC reflection is not assumed. A protobuf JSON renderer may omit an empty repeated field. Test list cardinality rather than textual JSON shape.

6.1 / DEPLOYMENT STRATEGY

From a reviewed change to an accepted release

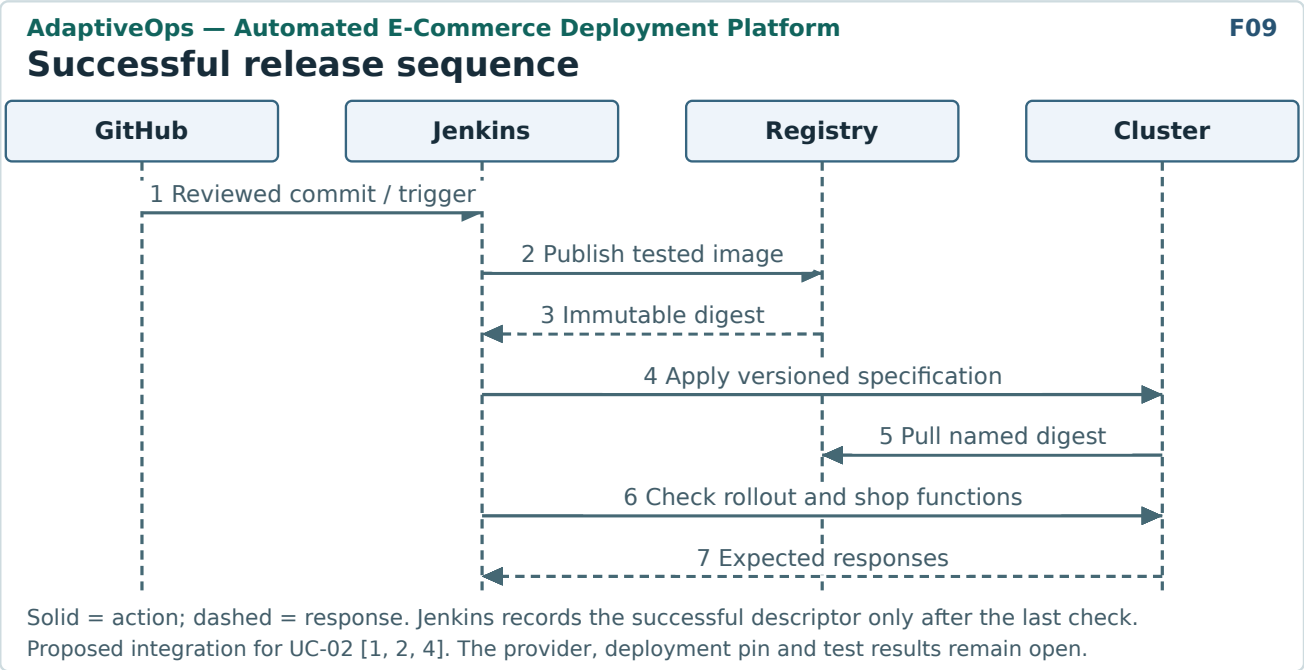


Figure 09 • A controlled trigger identifies one reviewed Git revision. Jenkins runs tests before publication, applies a versioned Kubernetes specification and runs functional checks against the deployed workload. An accepted release record is written only after rollout and smoke tests pass.

Build and promotion contract

The proposed Jenkinsfile separates PR validation from release: untrusted PR code receives no deployment credentials. A reviewed revision triggers the release job. A catalog unit test and a documented Docker build provide initial CI gates; a build or test failure prevents deployment. The chosen registry must support both publication by the agent and digest-based pulls from the Ubuntu cluster.

The release descriptor records the team commit, upstream SHA, Jenkins build, service-to-image digests, manifest/config revision and target environment. Versioned configuration references exclude secret values. A single deployment lock prevents concurrent jobs from changing the same target or overwriting its last accepted record.

Gates after deployment

The pipeline awaits Kubernetes rollout completion using a bounded timeout, then checks a known product, catalog search match/non-match, a same-session cart and mock checkout. A response mismatch, RPC error, unsuccessful rollout or bounded timeout fails the candidate. Only a passing descriptor replaces the last accepted one.

On the first installation there is no prior accepted release. Failure stops the job and retains evidence; automatic restoration is exercised only after a known-good baseline has been established.

6.2 / RELIABILITY DESIGN

Automatic rollback and recovery validation

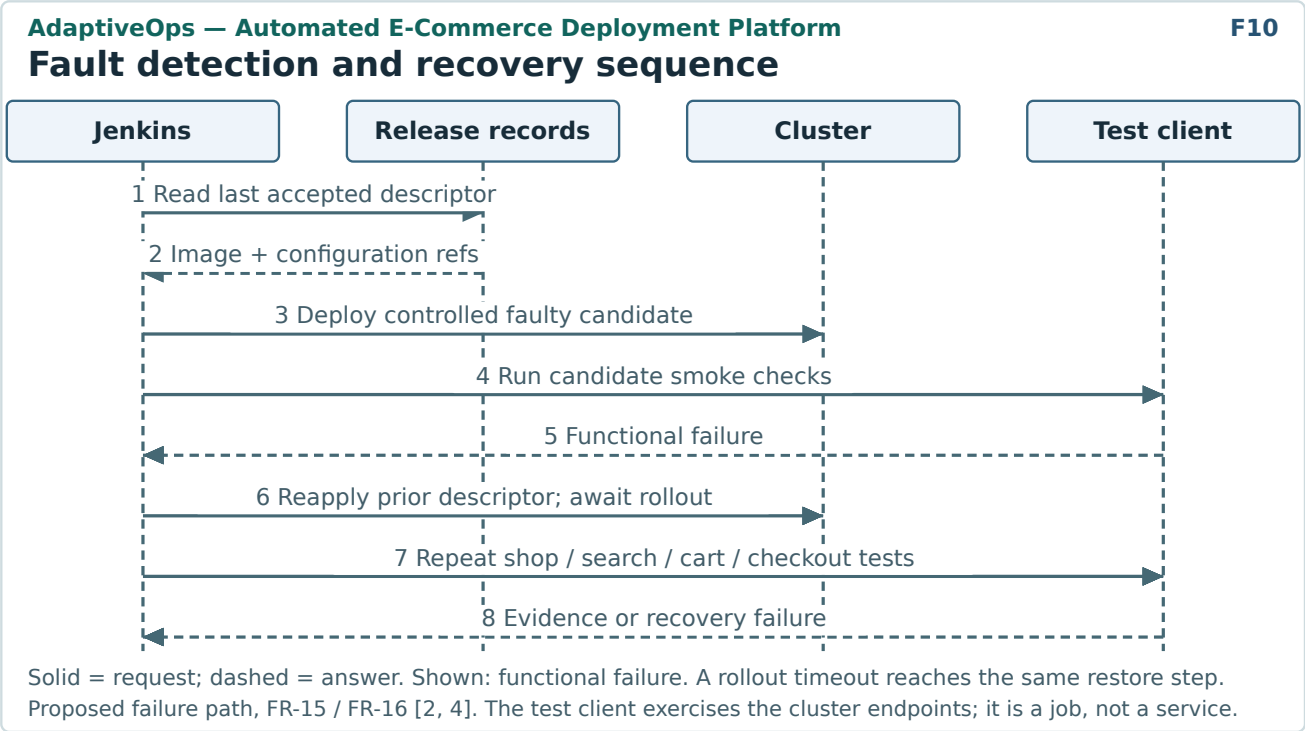


Figure 10 • The retained descriptor is read before an update. When a running candidate fails a real business operation, Jenkins reapplies the previously accepted image and matching configuration, waits for rollout and invokes the same checks. It records the failed candidate and recovery outcome separately.

Controlled fault scenario

A proposed demonstration changes only the candidate frontend's catalog address to an unreachable service in a disposable demo namespace. The static health handler can still respond, while product retrieval fails. A rollout timeout is a second supported failure path. If the gate fails, Jenkins restores the prior descriptor automatically; no operator selects a revision.

Conditions for declaring recovery

Ready Pods must serve the restored digest and configuration. The test client must again retrieve a known product, match a catalog query, read a newly added cart item and complete simulated checkout. Stage logs, timestamps, the rejected revision, both descriptors and redacted responses form one incident record.

If the accepted descriptor is unavailable, the restore operation times out, or any recovery check fails, the job ends as blocked. No further promotion or recursive rollback occurs. Application rollback does not restore Redis data, delete an order response, reverse a mock charge or revert unrelated infrastructure.

6.3 / SYSTEM BEHAVIOR

Release activities and states

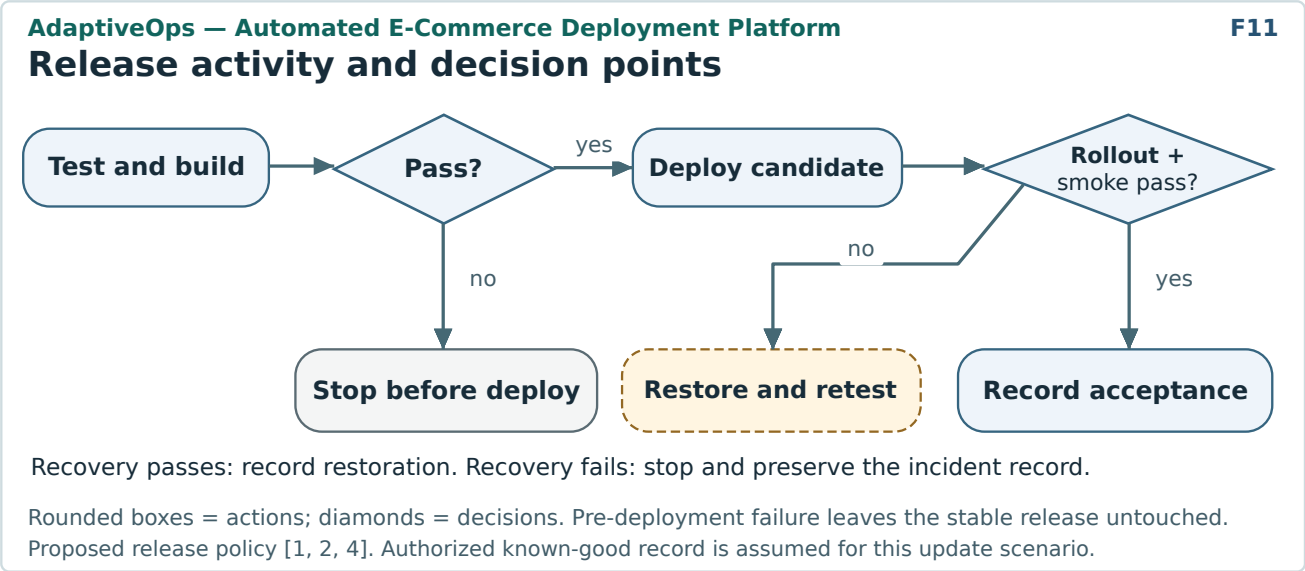


Figure 11 • Pre-deployment failures stop without changing the serving revision. Once a candidate is deployed, an unsuccessful gate triggers the restore-and-retest action. A successful candidate is recorded as accepted.

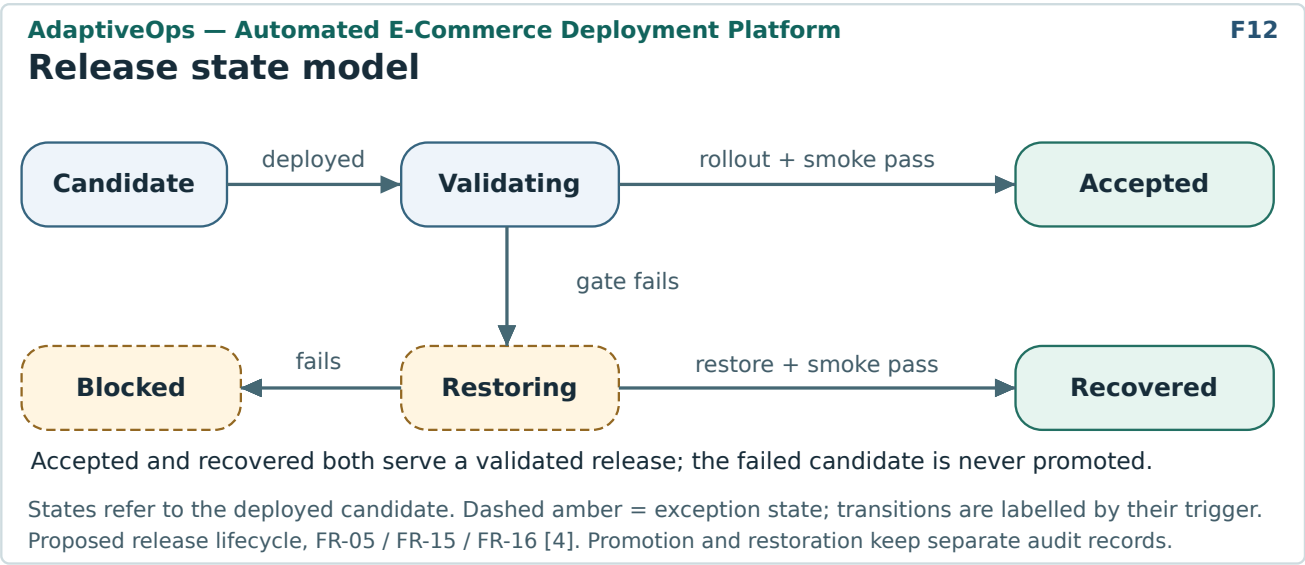


Figure 12 • The state belongs to the release attempt and its serving revision. Recovered means a previous version passed the checks again; the failed revision never becomes accepted. Blocked preserves the incident when the restore or its verification fails.

Release states are recorded by Jenkins. Kubernetes Deployment conditions supply rollout observations; the controller does not implement the full application-level recovery policy [14].

7.1 / UI AND UX DESIGN

Customer-facing storefront

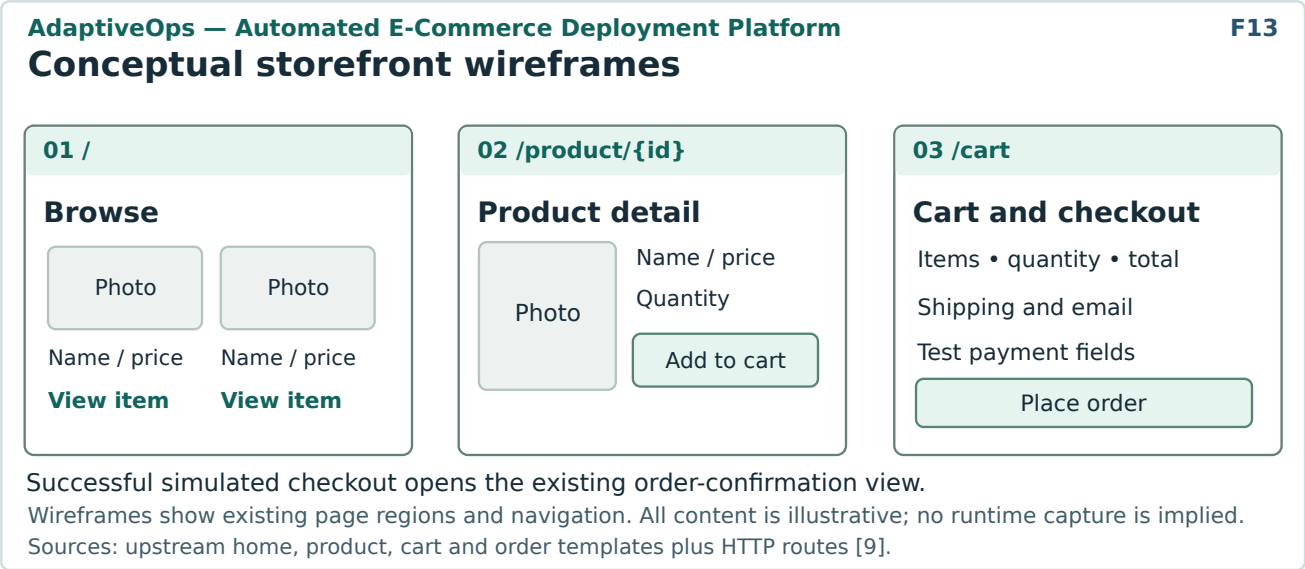


Figure 13 • The conceptual wireframes summarize three screens already rendered by the frontend: product listing, detail with quantity, and cart with checkout. A successful simulated order leads to the existing confirmation view. No new customer account, search form or order history is required.

Table 09 Existing interactions and review criteria

Interaction	Expected behavior and usability check
Browse and select	Product image, name and price identify the item. A product link opens the matching detail page. The selected item and price should remain readable at a narrow viewport.
Add and inspect	The quantity control and add action update the visitor's cart. A keyboard-only pass should reach both controls and the cart; a text label must identify the quantity field.
Checkout and confirmation	The existing form collects delivery/contact and test payment inputs. A validation error should remain distinguishable from a successful confirmation; redacted evidence excludes payment fields.
Catalog search	SearchProducts is tested through its service API. No website search control is drawn or added. The comparison uses records from the deployed catalog fixture.

The recommended review preserves the existing hierarchy and typeface. Labels should remain legible when zoomed, focus should be visible, and color should not be the sole cue for status or errors. Findings belong in the usability test record; no accessibility certification or frontend redesign is claimed.

7.2 / UI AND UX DESIGN

Operator dashboard and interpretation

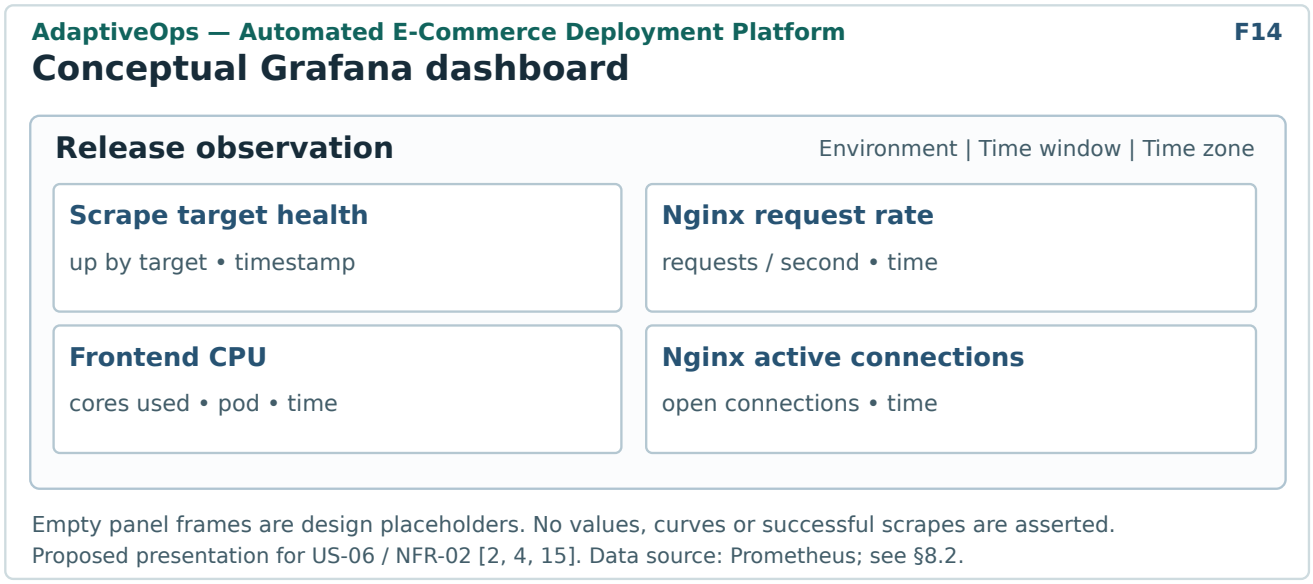


Figure 14 ● The proposed Grafana view places target health and traffic before resource use. Panel titles, units, filters and timestamps establish what each observation means. Empty panels denote the absence of measured data in this design, not zero traffic or successful monitoring.

Table 10 Dashboard definitions and data sources [15]

Panel and source	Meaning and limit
Scrape health ● Prometheus up	A successful scrape is evidence that a target responded. It does not establish a successful shop purchase. Missing samples should display “no data” distinctly from zero.
Request rate ● Nginx exporter	Proposed query: rate(nginx_http_requests_total[1m]) for the selected proxy target. Display requests/s. The interval is a display choice, not a performance threshold.
Frontend CPU ● kubelet/cAdvisor	Aggregate a verified container_cpu_usage_seconds_total rate for the frontend Pod/container. Display cores and time; filter duplicate/empty container labels after discovery.
Active connections ● Nginx exporter	Display nginx_connections_active for the selected target. Open connections are not completed transactions. Error rate and p95 come from controlled client runs unless separately instrumented.

The proposed layout uses labeled series, clear units and a shared time range. Show status text or symbols alongside color; retain keyboard access and readable type. Display a timestamp and release identifier with exported evidence so a panel can be compared to the load or fault run.

8.1 / DEPLOYMENT AND INTEGRATION

Local deployment topology

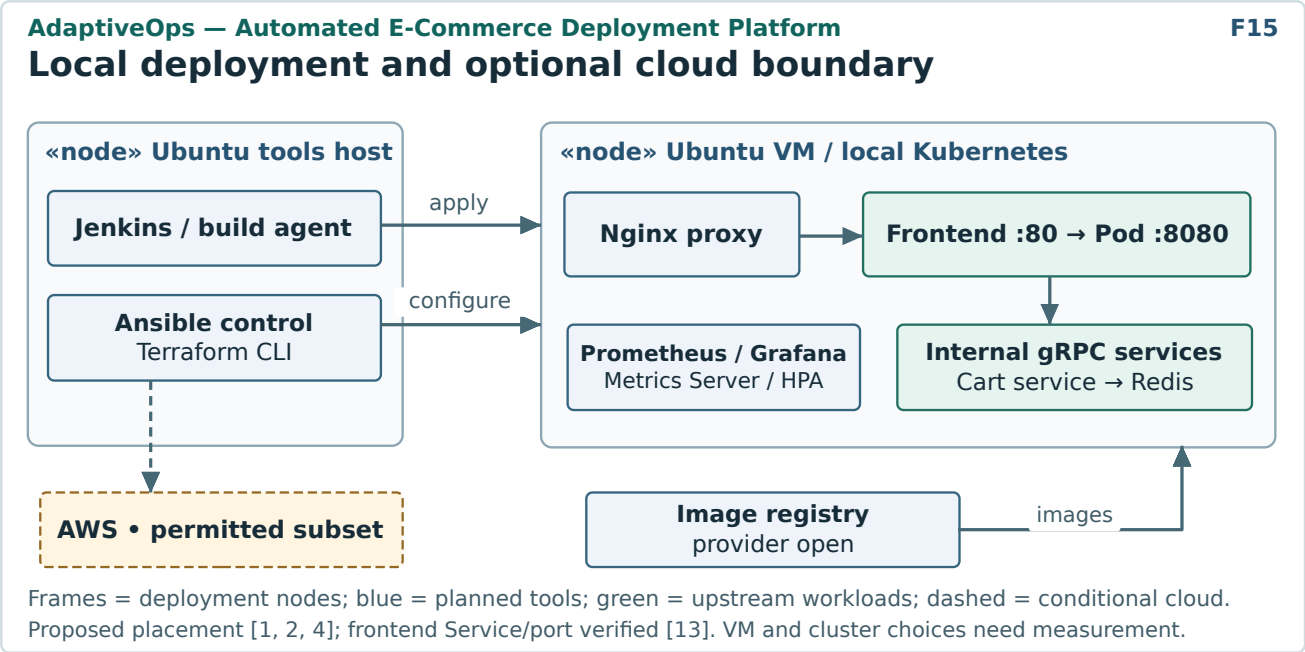


Figure 15 • The drawing separates logical tools and runtime hosts; they may share a measured Ubuntu guest or run on two team VMs. The proxy forwards to the frontend ClusterIP Service on port 80, targeting Pod port 8080. The internal services, Redis and operator endpoints remain private.

Cluster choice and entry route

Proposed default: a single-node kind cluster using Docker on Ubuntu, with an explicit host port mapping for an Nginx NodePort. For a NodePort mapping, the mapped node-container port must equal the chosen Service nodePort [14]. Bind the demo entry locally and use a controlled tunnel for remote review. Minikube with the Docker driver is a practical alternative if it fits the measured guest better. No distribution or registry has been adopted in the evidence.

The local overlay uses a configured Nginx Deployment and Service. Its upstream is frontend:80; the optional upstream frontend-external LoadBalancer is unnecessary on the local path. Kubelet pulls images from the selected registry by digest. Jenkins requires a reachable Kubernetes API and namespace-limited deployment permissions. Grafana access uses a private connection.

Capacity checkpoint

Approximately 16 GB host RAM and 100 GB disk were reported; the guest may receive less. Measure available guest RAM, vCPU, disk and image-cache growth, then check Pod requests, scheduling and headroom during a build and load run. If capacity is insufficient, move the Jenkins agent or monitoring to a second VM before repeating the same acceptance tests.

8.2 / SCALING AND OBSERVABILITY

Autoscaling and measurement

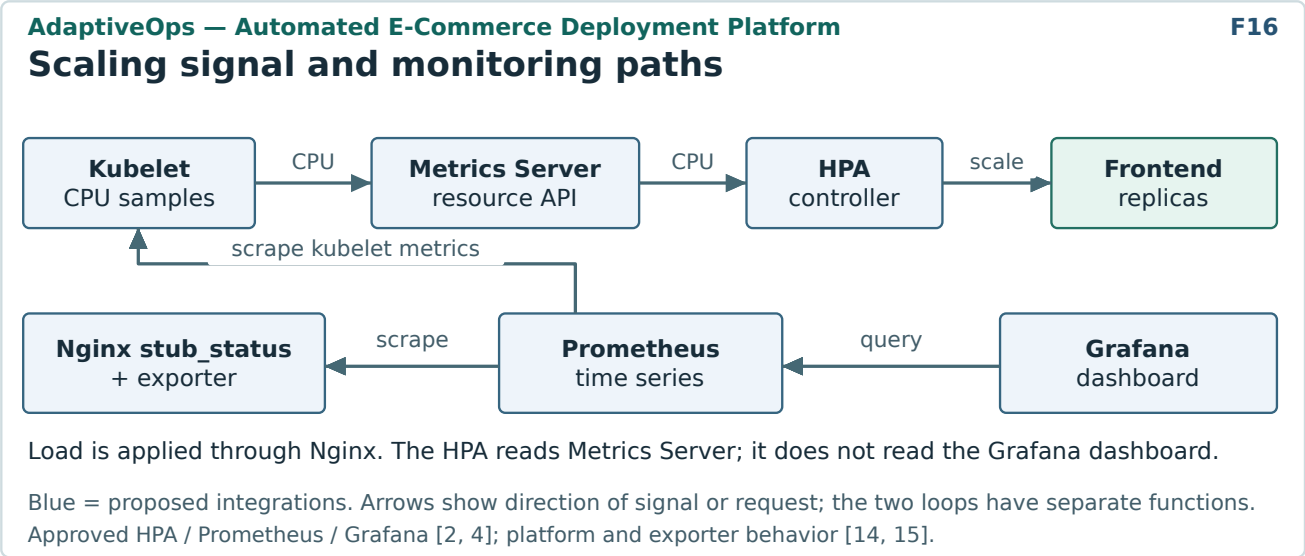


Figure 16 • The resource-metrics path supplies CPU data to the HPA controller. Prometheus separately scrapes targets for operator observation; Grafana queries Prometheus. One path cannot substitute for the other. Source manifests show CPU requests on the frontend, making it a candidate rather than a proven scaling target.

Controlled HPA experiment

Use autoscaling/v2 on the frontend Deployment, with minimum 1 and a tested maximum of at least 2 that fits the guest. Verify CPU requests for eligible containers, Metrics Server availability, the metrics.k8s.io API and HPA conditions before applying load. Choose the CPU utilization target and stabilization window after observing idle and loaded behavior. Avoid reapplying a fixed replica count over the active HPA.

Start from one ready replica. Fix the client profile, concurrency, duration and dataset. Point the existing Locust workload at Nginx, overriding its default frontend:80 target (including its readiness/init client). Record actual CPU, desired and ready replicas, and continued storefront responses. Stop the load and wait through stabilization. The approved criterion is 1 to at least 2 and back toward minimum; a traffic increase alone is insufficient.

Measured evidence

For comparable baseline/load windows, report requests per second, p95 response latency and error rate with request count, timestamps and guest allocation. Prometheus scrape health and the proposed private Nginx stub_status adapter are independent observations. Inspect exported metric names/labels and kubelet access before enabling panels. No p95, throughput or error threshold is fixed before feasibility [2, 4, 14, 15].

8.3 / CONFIGURATION AND SECURITY

Host configuration and conditional AWS scope

The Ubuntu deployment must remain reproducible without an AWS lab session. Ansible prepares that environment; Terraform expresses the feasible infrastructure subset required by the brief. These artifacts have different acceptance evidence.

Table 11 Configuration and infrastructure controls

Design element	Proposed mechanism	Evidence or boundary
Ansible inventory	Describe actual VM hosts, roles and non-secret version variables; keep environment overrides separate. Use declared package, file and service state.	One real host is configured; a second run produces no unintended changes. Retain inventory template and both recaps.
Terraform scripts	Version the provider lock file and module variables. Validate the configuration; inspect the plan, permissions, region, cost and cleanup path before any live use.	EC2 or S3 are candidates only. RDS/ELB are retained as brief items subject to necessity and feasibility. No EKS or SQL integration is required.
Live cloud gate	If an allowed resource is useful, use an isolated project scope. Record provisioned IDs privately, run bounded checks, then destroy and inspect residual resources.	If blocked, retain scripts and accurate validation/plan output; record the constraint. A plan is not an applied or cleaned-up resource.
Credentials and scope	Use restricted credentials for image publication and namespace deployment. Keep secret values out of Git, evidence and manifests; store only configuration templates.	Review role permissions, secret references and redaction; prevent PR validation from receiving release or cloud credentials.
Artifacts and cleanup	Keep references to validated images/configuration long enough for release recovery. Restrict registry, metrics and dashboard access to intended users.	Log versions and outcomes, not kubeconfigs, access tokens, card details or raw Terraform state. Restore only the affected release.

The lecturer-approved feasible-subset rule narrows live cloud usage, not the Terraform deliverable [1, 3, 7]. Whether live provisioning is possible is an implementation decision. No cloud resource was created as part of this design.

9.1 / TESTING AND VALIDATION

Functional and release validation plan

The following procedures specify the inputs, expected observations and evidence for implementation. None is reported as a team run. Use one release descriptor and an isolated test session per scenario; timestamp all evidence.

Table 12 Planned unit, integration and customer acceptance tests

Test and scope	Stimulus and expected observation	Retained evidence / owner
T01 • Reproducibility FR-02/04; NFR-01	Start twice from a documented clean target with the same upstream pin, image digests and config. Both runs must reach the same functional gates.	Two environment inventories, deployment transcripts, build reference and cluster states. Jana / Kirollos.
T02 • Storefront and cart FR-03/08; UC-01	Through Nginx, open the known product, add a fixed quantity and read the same session's cart. Complete simulated checkout with valid synthetic data; verify order result and redacted mock-payment evidence.	URL, selected product, same-session cart and sanitized order trace. Jana / Abdelrahman.
T03 • Catalog contract FR-03; US-02	Use pinned proto and fixture. Query "sunglasses" and a string absent from all names/descriptions. Compare returned IDs and require zero results for the non-match. Recheck fixture if pin changes.	Request/response, catalog ID and revision. Abdelrahman / Jana.
T04 • Unit, build, deploy FR-04–08; UC-02	Run catalog unit tests (e.g. TestSearchProducts), build a changed service image, publish the digest and deploy it. Verify ready Pods and functional smoke. A failed pre-deploy gate must not promote.	Commit/PR, test/build log, digest, rollout and smoke records. Amer / Kirollos.
T05 • Observability FR-09/10; NFR-02	Confirm each configured target exposes the selected series. Observe the same time window in Prometheus and Grafana. Distinguish missing data from zero and scrape success from business health.	Target status, raw series and live dashboard capture. Rahma.

Acceptance boundary

The user-facing check must inspect returned content. An HTTP 200 alone does not prove that the catalog item is present, that a cart was stored, or that checkout completed. Any mismatch preserves the actual result and fails the relevant gate.

9.2 / TESTING AND VALIDATION

Scaling, recovery and operational acceptance

Table 13 Planned resilience, infrastructure and peer acceptance tests

Test and scope	Stimulus and expected observation	Retained evidence / owner
T06 • HPA and access FR-11/12; NFR-04	Verify resource metrics and a one-replica start. Apply the fixed profile through Nginx, observe at least two replicas and client access; remove load and observe reduction toward minimum.	Timestamped load, CPU, HPA/Pod timeline and client log. Kirollos / Rahma / Abdelrahman.
T07 • Fault and recovery FR-15/16; UC-03	After T06 returns toward baseline, introduce the controlled bad release. A failed rollout or functional gate must initiate restoration without manual version selection. Compare restored digest/config and rerun T02/T03.	Failure log, gate decision, old/new descriptors, restored Pods and redacted recovery results. Amer / Abdelrahman.
T08 • Performance NFR-10	Compare baseline and load over defined windows, workload and VM allocation. Compute throughput, p95 latency and errors; set performance thresholds only after feasibility.	Client report, sample counts, duration and test settings. Rahma / Abdelrahman.
T09 • Host setup FR-13; NFR-06	Configure the declared Ubuntu host; run Ansible again using identical inputs. Review unexpected changes and resulting service state.	Two playbook recaps, host state and versioned inventory template. Rana.
T10 • Feasible cloud FR-14; NFR-07	Validate scripts; check price, authorization and cleanup. If live use is permitted, inspect the bounded plan, apply, test and destroy. If unavailable, record the exact stopping point.	Validation/plan, conditional apply/destroy and residual-inventory evidence. Rana.
T11 • Runbook and controls FR-01/17; NFR-05/08/09	A peer follows the local runbook without an AWS session. Review secrets, permissions, attribution, each member's contribution and limitations.	Independent checklist, issues, versioned docs and evidence links. Abdelrahman / all six.

The six planning KPIs map to T01 (two clean deployments), T04 (test/build/controlled deployment), T06 (HPA), T05 (live dashboards), T07 (automatic recovery) and T09 (repeatable Ansible). Failed or blocked checks remain in the test record.

10.1 / OWNERSHIP AND DELIVERY

Team responsibilities and project milestones

Table 14 Approved responsibilities and review subjects [1, 2]

Member • approved ownership	Hands-on collaboration and design review
Amer Hossam Eldin Mohamed Mustafa (Team leader) Jenkins CI/CD, release integration and rollback.	Docker, Kubernetes and monitoring. Reviews the release descriptor, decision gates, automatic restore and integration evidence (F02, F09–F12).
Kirollos Magdy Zekry Awad Kubernetes, Nginx, HPA and networking.	Jenkins and Ansible. Reviews cluster placement, Service routing, resource requests and the scaling signal (F15–F16).
Rana Ashraf Abu Elyazeed Abu Elyazeed Ansible, Terraform and AWS feasibility.	Kubernetes and manifests. Reviews host inventory, repeatability, permitted cloud scope and cleanup (§8.3).
Rahma Mohamed Abdo El-Sharqawy Prometheus, Grafana, KPIs and metrics.	Docker, HPA and testing. Reviews scrape targets, panel definitions and timestamps (F14, F16).
Jana Mohamed Mahmoud El-Waleely Application feasibility, Docker builds and reproducible startup.	CI and Kubernetes. Reviews service dependencies, source attribution, fixtures, data storage and customer flow (F03–F05, F08, F13).
Abdelrahman Ahmed Abdelbaset Mohamed Smoke, load and rollback tests; execution documentation.	Jenkins and cluster operations. Reviews actors, scenario coverage, test evidence and runbook usability (F01, F06–F10).

Delivery sequence

16 October: planning, review and requirements baseline. 6 November: system analysis and design. Weeks 4–5 (7–20 November): integrate CI/CD and Kubernetes, then monitoring and scaling. Week 6 (21–27 November): controlled failure, recovery and integration. Week 7 (28–30 November): stabilize and deliver implementation, source and execution guide [1, 2, 5].

4 December remains the separate deadline for final testing, reports and presentation. Commit, test or walkthrough evidence should identify the actual contribution of each member. Design ownership does not certify that an activity has been performed.

10.2 / AUDIT AND DECISION REGISTER

Discrepancies, assumptions and open evidence

Table 15 Issue, sources and documented treatment

Issue and sources	Safe design treatment	Disposition / owner
ERD, keys and normalization • DEPI [5] versus approved model [3, 4] and source [9–11]	F04 and Table 05 document real entities, attributes, identifiers, relationships and physical storage. SQL tables and relational normalization are inapplicable.	Format acceptance of the non-relational substitute remains open; Amer / Jana.
“Search” and “database” • Project 4 [7] versus approved scope [1, 3, 4]	SearchProducts is tested at the gRPC API. Redis provides cart storage with emptyDir; JSON supplies default catalog data. Neither a browser search feature nor relational orders is assumed.	Scope established; verify behavior on the team’s pin. Jana / Abdelrahman.
Four AWS services in the brief [7] versus feasibility allowance [1, 3]	Keep Terraform scripts and validation; EC2/S3 are candidates. RDS/ELB are conditional and require technical justification.	Actual subset, permissions, costs and allowable live evidence are unverified. Rana.
Source audit SHA versus team deployment [3, 8]	The inspected revision identifies facts in this report. It does not select a release for implementation. Recheck routes, proto, images and storage on the adopted pin.	Deployment pin and reachable image registry still open. Jana / Amer.
Capacity, signals, thresholds [2, 4]	Measure the guest. Trial the proposed kind layout, Prometheus targets and frontend CPU scaling. Calibrate timeouts and load after feasibility.	Guest allocation, viable HPA target, exporter series and performance limits untested. Kirollos / Rahma.
Submission responsibility [6, 17]	The leader coordinates final delivery; the handoff identifies Kirollos and Jana as teammates with submission access. The assigned repository remains the destination.	No November upload claimed; authorized teammate may submit after review.
Approval versus runtime [1–4, 16]	The October documents and approved plan are verified as records. No reviewed artifact establishes a team deployment, dashboard measurement, rollback run or AWS resource.	Runtime evidence is outstanding; absence from the October files does not rule out local work.

The grading rubric was not supplied [3]; no marks or lecturer quotations are inferred. The release, registry, HPA and cloud decisions above are implementation choices within the approved plan, not additional project features.

APPENDIX A / COMPLIANCE

DEPI Section 4 coverage

The checklist follows the DEPI column and Section 4 of R5 Project Documentation Last Updates, pp. 1–3 [5]. Each entry identifies the design artifact or the specific limitation. “Covered” denotes documentation coverage only.

Table 16 Official design deliverables and report locations

Official item	Report location	Treatment
1a Problem statement and objectives	§1 • p. 3	Covered
1b Use case diagram and descriptions	§2 • p. 4 • F01	Covered
1c Functional / non-functional requirements	App. B • pp. 25–27	Covered
1d Software architecture	§3 • pp. 5–6 • F02–F03	Covered
2a Entity–relationship diagram	§4.1 • p. 7 • F04	Conceptual ERD; format clarification
2b Logical and physical schema	§4.1–4.2 • pp. 7–8	Covered for Redis/JSON; SQL N/A
3a Context and detailed DFD	§5.1–5.2 • pp. 9–10 • F06–F07	Covered
3b Sequence diagrams	§5.3, §6.1–6.2 • pp. 11–13 • F08–F10	Covered
3c Activity diagram	§6.3 • p. 14 • F11	Covered
3d State diagram	§6.3 • p. 14 • F12	Covered
3e Class diagram	§4.2 • p. 8 • F05	Covered: inspected C# types
4a Wireframes and mockups	§7 • pp. 15–16 • F13–F14	Covered: conceptual views
4b UI/UX guidelines	§7 • pp. 15–16	Covered
5a Technology stack	§3, §8 • pp. 5–6, 17–19	Covered
5b Deployment diagram	§8.1 • p. 17 • F15	Covered
5c Component diagram	§3.2 • p. 6 • F03	Covered
6a API documentation	§5.3 • p. 11 • Table 08	Applicable; covered
6b Testing and validation	§9 • pp. 20–21 • T01–T11	Applicable; covered
6c Deployment strategy and scaling	§6, §8 • pp. 12–14, 17–19	Applicable; covered

The source-derived class diagram is applicable. A conventional relational physical ERD would be misleading. The supplied conceptual ERD and non-relational schema describe the existing system directly; the outstanding question concerns presentation format only.

APPENDIX B / REQUIREMENTS TRACEABILITY

Functional requirements FR-01 to FR-08

These requirements establish source provenance, a reproducible build and the path from a controlled change to the storefront. A / NT = approved [4]; not yet tested. Owners follow [1, 2].

Table 17 Traceability by approved identifier

ID / owner / status	Approved requirement or story	Design mechanism and location	Intended verification
FR-01 Amer / Jana A / NT	Keep team code, configurations and documents in the assigned Git repository with source attribution.	Reviewed branches and a license/upstream revision record. §1; §6 / F09	Inspect PR, commit, Apache-2.0 notice and repository paths.
FR-02 Jana / Kirollos A / NT	Reproduce deployment of an explicitly pinned Online Boutique revision.	Record upstream SHA, team revision, exact images and manifest settings. §6; §8 / F15	Run the same descriptor twice from clean targets; compare.
FR-03 Jana / Abdelrahman A / NT	Verify storefront, SearchProducts, mock payment and Redis cart; document scope and storage limits.	Browse and checkout through Nginx; test catalog gRPC and same-session cart separately. §4–5 / F04, F08	Capture matched/nonmatched queries, cart contents and simulated order.
FR-04 Jana / Amer A / NT	Build or adapt container images using documented Docker procedures.	Use the pinned service Dockerfile, build context and recorded build inputs. §6 / F09	Repeat the documented build; retain image identifier and build log.
FR-05 Amer / Abdelrahman A / NT	Run automated Jenkins CI tests and an image build on a controlled trigger.	Run trusted reviewed revision through test and build; stop on failure. §6 / F09, F11	Compare successful and failed stage logs to their commits.
FR-06 Amer / Jana A / NT	Publish versioned images or artifacts with metadata identifying the release.	Publish a commit-labelled image; retain the registry digest and config ref. §6 / F09	Resolve published digest to build, team commit and deployment record.
FR-07 Kirollos / Jana A / NT	Deploy application and dependencies to Kubernetes using versioned manifests, chart or configuration.	Render and retain local overlays with digest-pinned Deployments and Services. §8 / F15	Compare rendered manifest to rollout, Pods and Service endpoints.
FR-08 Kirollos A / NT	Expose the application through a configured Nginx reverse proxy or routing entry.	Route the VM entry to Nginx and then to the internal frontend Service. §8.1 / F15	Reach a known product through the proxy; match access log to request.

APPENDIX B / REQUIREMENTS TRACEABILITY

Functional requirements FR-09 to FR-17

Monitoring, scaling, infrastructure scripts, automatic recovery and project evidence retain their original scope. FR-14 preserves its conditional apply. A / NT = approved [4]; not yet tested. Owners follow [1, 2].

Table 18 Traceability by approved identifier

ID / owner / status	Approved requirement or story	Design mechanism and location	Intended verification
FR-09 Rahma / Kirollos A / NT	Collect selected application or infrastructure metrics in Prometheus.	Scrape a configured Nginx exporter and authorized kubelet series. §8.2 / F16	Inspect target discovery, up, sample names, labels and timestamps.
FR-10 Rahma A / NT	Provide Grafana dashboards sourced from Prometheus to display measured trends.	Bind each panel to a tested query with a named unit and time range. §7.2 / F14; §8.2	Export dashboard and capture live timestamped samples.
FR-11 Kirollos / Rahma A / NT	Configure HPA with valid metric sources and scale-out/down controls.	Use frontend CPU requests, Metrics Server and autoscaling/v2; bound replicas. §8.2 / F16	Check metrics API, HPA conditions and actual replica transitions.
FR-12 Abdelrahman / Kirollos A / NT	Generate repeatable load; record actual scaling and client access.	Fix load profile and duration; probe storefront before, during and after load. §8.2; §9 / T06	Save client outcomes, load parameters and HPA timeline.
FR-13 Rana A / NT	Configure hosts or services with reproducible Ansible playbooks.	Use inventory, versioned roles, declared packages and controlled service configuration. §8.3; §9 / T09	Run playbook twice; explain changes and compare host state.
FR-14 Rana / Amer A / NT	Provide and validate feasible AWS Terraform scripts with cost checks and cleanup; apply only when available and authorized.	Keep a bounded module, variables, validation and teardown path; gate live use. §8.3; §9 / T10	Archive validation; where permitted attach plan, price/permission review and cleanup.
FR-15 Amer / Kirollos A / NT	Detect a defined rollout or functional failure and automatically restore the retained known-good image and configuration.	Jenkins selects the last accepted descriptor and reapplies it without manual version selection. §6.2–6.3 / F10–F12	Inject one bad candidate; compare pre-fault and restored digest and config.
FR-16 Abdelrahman / Amer A / NT	Verify recovery with functional smoke tests; preserve failure/rollback logs and configuration or data limits.	Rerun shop, gRPC search, cart and checkout on the restored release. §6.2; §9 / T07	Save both traces and limits; a failed recovery stays blocked.
FR-17 Abdelrahman / all six A / NT	Record setup, environment prerequisites, team contributions, tests and known limitations.	Version runbook and evidence index; connect each contribution to a change or test. §9–10 / T11	Independent teammate follows setup and checks linked records.

APPENDIX B / REQUIREMENTS TRACEABILITY

Non-functional requirements NFR-01 to NFR-10

The approved acceptance policies are retained. Performance measurements are required; numerical thresholds are deferred until feasibility. A / NT = approved [4]; not yet tested. Owners follow [1, 2].

Table 19 Traceability by approved identifier

ID / owner / status	Approved requirement or story	Design mechanism and location	Intended verification
NFR-01 Jana / Kirollos A / NT	Reproducibility: the same documented configuration deploys twice from a clean target.	Reset a disposable target; reuse the same descriptor and instructions. §9 / T01, T04	Two dated installation/deployment traces and cluster states.
NFR-02 Rahma A / NT	Observability: at least one Grafana dashboard shows live timestamped metrics and visible target health.	Display scrape health alongside request rate and resource use. §7.2; §8.2 / F14, F16	Prometheus target status and timestamped Grafana capture.
NFR-03 Amer / Abdelrahman A / NT	Reliability: unhealthy releases trigger rollback and functional recovery checks; a static health endpoint alone is insufficient.	Combine rollout and storefront/service checks; automatically restore. §6.2 / F10; T07	Fault injection, restore record and functional recovery responses.
NFR-04 Kirollos / Rahma A / NT	Scalability: one eligible workload scales from 1 to at least 2, then toward minimum after load ends.	Use a feasible CPU target with a single-replica starting point and measured load. §8.2 / F16; T06	Timeline of load, resource metric, desired/ready replicas and client access.
NFR-05 Amer / Rana A / NT	Security: no committed secrets; use environment/Secret/config templates and least privilege.	Separate build, namespace deployment, monitoring and cloud credentials. §6.1; §8.3	Inspect Git, manifests, role permissions and sanitized evidence.
NFR-06 Rana / Kirollos A / NT	Maintainability: version, comment where useful, and support per-environment configuration in scripts.	Separate shared base from local overrides; document parameters and pin versions. §6.1; §8.3	Review diffs and rerun with the documented environment inputs.
NFR-07 Rana A / NT	Cost control: check price and permission; document cloud destruction and cleanup.	Restrict Terraform scope and preserve removal/inventory evidence after any live use. §8.3 / T10	Cost/permission review, plan and teardown record if applied.
NFR-08 Kirollos / Jana A / NT	Portability: the main demo runs on a local Ubuntu VM independently of AWS sessions.	Place shop, cluster, monitoring and recovery locally; no live AWS dependency. §8.1 / F15; T04	Demonstrate the local route and deployment with no AWS session.
NFR-09 Abdelrahman / peer A / NT	Usability: another teammate follows the README/runbook and reproduces a known deployment.	Provide prerequisites, commands, expected checks, cleanup and failure notes. §9 / T11	Peer execution checklist with discrepancies and corrections.
NFR-10 Rahma / Abdelrahman A / NT	Performance: record throughput, p95 latency and error rate for baseline/load runs; agree thresholds after feasibility.	Fix workload/duration and guest resources; distinguish client metrics from scrape health. §8.2; §9 / T06, T08	Timestamped client reports, parameters and observed limits; no preset SLO.

APPENDIX B / REQUIREMENTS TRACEABILITY

User stories US-01 to US-09

Customer, tester, contributor, release, monitoring and infrastructure perspectives are preserved. API-level search is separate from the browser workflow. A / NT = approved [4]; not yet tested. Owners follow [1, 2].

Table 20 Traceability by approved identifier

ID / owner / status	Approved requirement or story	Design mechanism and location	Intended verification
US-01 Jana / Kirollos A / NT	Customer: reach the deployed shop and see products.	Browse the Nginx URL and open a catalog item from the grid. §2; §7.1 / F13	Accessible URL and correct product page.
US-02 Abdelrahman / Jana A / NT	Tester: call SearchProducts and check the returned records against the sample catalog.	Send known-match and non-match queries against the internal gRPC API. §5.3; §9 / T03	Requests, responses and matching catalog fixture.
US-03 Jana / Abdelrahman A / NT	Customer: add a cart item and simulate checkout without real payment.	Reuse a single session; submit the existing checkout form with synthetic inputs. §4; §5.3 / F08	Cart quantity, mock transaction evidence and order response.
US-04 Amer / Jana A / NT	Contributor: submit a change through automated checks and a container build.	Reviewed Git change identifies the revision tested and built by Jenkins. §6.1 / F09	Commit/PR and test/build console output.
US-05 Amer / Kirollos A / NT	Release operator: deploy a known version and determine whether it is healthy.	Observe rollout and functional gates for an immutable image/config pair. §6 / F09, F11	Image reference, rollout state and smoke log.
US-06 Rahma A / NT	SRE: observe workload health and demand on dashboards.	Use Prometheus-backed panels for target health, traffic and frontend CPU. §7.2; §8.2 / F14, F16	Live series, target status and dashboard view.
US-07 Kirollos / Abdelrahman A / NT	SRE: generate load and observe automatic replica scaling.	Replay fixed traffic through Nginx; correlate HPA decisions with client load. §8.2; §9 / T06	Load log and replica progression through recovery toward minimum.
US-08 Amer / Abdelrahman A / NT	SRE: observe a faulty deployment being detected and automatically reverted.	Jenkins restores the accepted descriptor when the release gate fails. §6.2 / F10; T07	Failure, selected revision, rollback log and successful recovery checks.
US-09 Rana A / NT	Infrastructure operator: reproduce host or optional AWS setup from versioned code.	Ansible configures the host; approved Terraform subset has a conditional apply path. §8.3; §9 / T09, T10	Playbook output; feasible plan and safe resource evidence when applicable.

APPENDIX B / REQUIREMENTS TRACEABILITY

Use cases and the nine official deliverables

All three October use cases retain their original event order and intended outcome. A / NT = approved [4]; not yet tested. Owners follow [1, 2].

Table 21 Traceability by approved identifier

ID / owner / status	Approved requirement or story	Design mechanism and location	Intended verification
UC-01 Jana / Abdelrahman A / NT	Customer demonstration: visit the storefront, inspect product details, add to cart and complete simulated checkout; verify page, cart and mock payment.	Use a synthetic session on the Nginx route and actual upstream services. §2; §5.3 / F01, F08	Product/quantity, redacted mock-payment record and order confirmation.
UC-02 Amer / Kirollos A / NT	Automated deployment: reviewed change → Jenkins checks and image build → Kubernetes deploy → smoke tests; accept only after validation.	On the controlled trigger, publish by digest; promote only after all gates. §2; §6.1 / F01, F09, F11	Commit, image digest, passing stage/rollout logs and release record.
UC-03 Kirollos / Amer / Abdelrahman A / NT	Reliability demonstration: apply load and verify HPA, then deploy a controlled fault, restore the known-good release and repeat smoke tests.	Separate the load interval from the fault exercise; record both on one timeline. §2; §6.2; §8.2 / F10, F16	Replica and CPU timeline, bad release ID, restore and functional recovery evidence.

Table 22 Original brief crosswalk

Project 4 deliverable [7]	Design / verification link
1 Dockerized microservices	FR-02–04 • F03, T01–T04
2 CI/CD update pipelines	FR-05/06/15/16 • F09–F12
3 Kubernetes with autoscaling	FR-07/11/12 • F15–F16, T06
4 Ansible server configuration	FR-13 • §8.3, T09
5 Prometheus monitoring dashboards	FR-09/10 • Prometheus + Grafana; F14, T05
6 Nginx reverse proxy	FR-08 • F15, T02
7 Terraform scripts for AWS resources	FR-14 • feasible subset; §8.3, T10
8 Automated rollback	FR-15/16 • F10–F12, T07
9 Git repository for code and scripts	FR-01/06/17 • §6.1, T11

APPENDIX C / REFERENCES

Approved sources and repository evidence

Documents [1]–[7] were read in full, including the Project 4 brief image. The four October PDFs establish the baseline. The November master prompt and coverage guide [17] direct this design package; they do not replace the approved sources.

[1] AdaptiveOps team, Project Proposal. October 2026. 3 pp. Approved PDF at repository snapshot

[2] AdaptiveOps team, Project Planning and Management. October 2026. 2 pp. Approved PDF at repository snapshot

[3] AdaptiveOps team, Literature Review and Lecturer Evaluation. October 2026. 2 pp. Approved PDF at repository snapshot

[4] AdaptiveOps team, Requirements Gathering. October 2026. 3 pp. Approved PDF at repository snapshot

[5] DEPI, R5 Project Documentation Last Updates. Supplied official guideline, 5 pp. Schedule: p. 1, DEPI column; System Analysis & Design: §4, pp. 2–3.

[6] DEPI, R5 Projects Instructions Final for Trainees. Supplied official trainee instructions, 3 pp.; group responsibility, company repository and final evidence.

[7] DEPI, Project 4: Automated E-Commerce Deployment Platform. Supplied official brief, Project_4_Brief.jpeg; problem, description, nine deliverables and tools.

[8] GoogleCloudPlatform, Online Boutique. Upstream reference revision 38e7348eb289eb5b87c0c6e8cb19ced0449dc389. Apache-2.0. This SHA identifies inspected code, not a team deployment. Repository at revision · README · License

Repository identity check

[16] AdaptiveOps team, official GitHub repository. Read-only inspection, 5 November 2026. Four local PDF byte streams matched the four remote Git blob IDs. Main snapshot: dab2f1f7fa908c7631b4c503b4677b53d892ed05. Snapshot

[17] AdaptiveOps team, Work master prompt and requirements lock v1.0. Supplied 5 November 2026 in the system design handoff. The standalone and packaged master prompt were identical.

The companion source inventory lists all inspected file paths and full identifiers. The source review does not establish the contents of teammates' local VMs.

APPENDIX C / TECHNICAL REFERENCES

Source code and engineering references

References [9]–[13] point to files at the immutable Google source revision on p. 30. Links are grouped by the claim they support. Online technical references were consulted on 5 November 2026.

[9] Google, API and storefront. Protobuf messages, gRPC methods, HTTP routing and the rendered store. demo.proto · routes · handlers · RPC client · home · product · cart · order

[10] Google, product catalog. JSON representation, loader selection and case-insensitive matching on product name or description. sample data · loader · methods · unit tests

[11] Google, cart service. The gRPC adapter, injected interface, cache implementation and provider registration establish the class and storage models. CartService.cs · ICartStore.cs · RedisCartStore.cs · Startup.cs · Cart.proto

[12] Google, checkout and payment. Order orchestration and simulated charge validation/transaction ID. PlaceOrder · charge.js

[13] Google, deployment and traffic. Services, probes, CPU requests, Redis volume, load target and recommendation-to-catalog call. frontend · cart / Redis · catalog · checkout · load manifest · Locust profile · recommendation client

[14] Kubernetes and kind documentation. HPA metrics, rollout status and rollback scope, Pod volume lifetime, and local port mapping. Horizontal Pod Autoscaling · Resource metrics pipeline · Deployments · emptyDir · kind configuration

[15] Prometheus, Grafana Labs and NGINX. Target health, Prometheus data-source configuration and Nginx OSS exporter measurements. Jobs and instances · Prometheus data source · NGINX Prometheus Exporter

Companion materials

The editable figure set contains F01–F16 in SVG. The Markdown traceability appendix exposes separate fields for ID, wording, mechanism, location, verification, owner and status. The audit and source inventory document the input checks and remaining design exceptions.